A Survey on How Test Flakiness Affects Developers and What Support They Need To Address It

Martin Gruber
BMW Group, University of Passau
Munich, Germany
martin.gr.gruber@bmw.de

Gordon Fraser
University of Passau
Passau, Germany
gordon.fraser@uni-passau.de

2022 IEEE Conference on Software Testing, Verification and Validation (ICST) | 978-1-6654-6679-0/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICST53961.2022.00020

***Abstract*—Non-deterministically passing and failing test cases, so-called *flaky* tests, have recently become a focus area of software engineering research. While this research focus has been met with some enthusiastic endorsement from industry, prior work nevertheless mostly studied flakiness using a code-centric approach by mining software repositories. What data extracted from software repositories cannot tell us, however, is how developers perceive flakiness: How prevalent is test flakiness in developers' daily routine, how does it affect them, and most importantly: What do they want us researchers to do about it? To answer these questions, we surveyed 335 professional software developers and testers in different domains. The survey respondents confirm that flaky tests are a common and serious problem, thus reinforcing ongoing research on flaky test detection. Developers are less worried about the computational costs caused by re-running tests and more about the loss of trust in the test outcomes. Therefore, they would like to have IDE plugins to detect flaky code as well as better visualizations of the problem, particularly dashboards showing test outcomes over time; they also wish for more training and information on flakiness. These important aspects will require the attention of researchers as well as tool developers.**

***Index Terms*—Flaky Tests; Empirical Study; Survey**

I. Introduction

Flaky tests are tests that behave non-deterministically, so they may both pass and fail when executed on the same code under test repeatedly. The problem of flaky tests has been extensively researched in the recent past, but prior work [1] primarily used a code-centric approach, providing only limited insights into how test flakiness affects developers. Furthermore, the few human studies that do exist never asked their participants directly about their wishes towards researchers and tool developers.

In order to address these limitations, we aim to complement prior research by finding out from developers in different fields directly how they experience flakiness, what they have already tried to mitigate it, and what tools or information they would desire to better handle this challenge. We surveyed 335 professional developers on their experiences with flaky tests and what they desire from the research community. Our sample population consists of 233 professionals from the general public all over the globe, as well as 102 employees of the BMW group. Using this setup, we are able to investigate flakiness in both a specific domain and software development in general. In detail, by asking this large set of developers, we aim to shed light on the following questions:

Prevalence and Severity: We asked participants how they experience flakiness. Developers perceive flakiness as a common and severe issue, with $51\,\%$ of all participants experiencing it at least weekly and $66\,\%$ rating it as a moderate or serious problem. Despite ranking flakiness a less severe issue, mobile application developers experience flaky failures very often due to flaky emulators and testing tools. On the other hand, safety-relevant software running in cars sees flaky failures less often but understandably considers it a more severe problem. Overall, flakiness is perceived as an equally or more pressing challenge in the automotive domain than in other domains.

Root Causes: We asked participants what causes of flakiness they have experienced. While concurrency-related issues remain overall most common, we observed differences between domains and also discovered uninitialized variables, compiler differences, and undocumented API changes to cause test flakiness—three previously undocumented categories.

Consequences: We asked participants how flakiness affects them. Losing trust and wasting developer time are perceived as the most severe impact, particularly more than the waste of computational resources through test reruns. Furthermore, we showed that the trust developers put in a test outcome can also depend on the outcome itself, which is important for tool developers who are already building on this assumption [2].

Mitigations: When asked how they currently handle flaky tests, developers responded that they mostly fall back to rerunning and rewriting test cases with little use of automated techniques; this is in part because there is a lack of practical tools for many development environments.

Wishes: Finally, we asked developers directly what tools or information they would like to possess to better deal with flakiness. They wish to use more sophisticated technologies such as IDE plugins that can statically detect flaky tests, as well as better visualizations of flakiness, like showing the outcomes of a test case over time. Lastly, they want more educational opportunities such as examples, guides, and best practices on how to deal with flaky tests. This knowledge should help tool developers to achieve a higher impact of their work.

It is our hope that these insights help to build a clear vision of how future research on test flakiness can create value for developers. We provide all data freely, allowing other researchers to verify, replicate, and extend our findings: https://doi.org/10.6084/m9.figshare.16727251

978-1-6654-6679-0/22/$31.00 ©2022 IEEE
DOI 10.1109/ICST53961.2022.00020

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

II. BACKGROUND

A. Prevalence and Severity

Scientific studies have found $0.5\,\%$ [3] to $1\,\%$ [4] of tests to be flaky. Practitioners at Google report a similar diffusion of flakiness among small tests [5] and a much stronger prevalence in larger ones: Overall, almost $16\,\%$ of their tests show some kind of flakiness, and $84\,\%$ of the transitions from pass to fail involve a flaky test [6]; out of $115\,160$ test targets that had previously passed and failed at least once, $41\,\%$ were flaky [7]. These technical metrics, however, do not necessarily reflect how often developers have to deal with the problem.

At the time of our study, there was only one other survey asking developers directly about their experience with flakiness, which involved 121 developers and was conducted by Eck et al. [8] in 2019. Of their participants, $58\,\%$ indicated that they have to deal with flaky tests at least monthly. Out of the $91\,\%$ experiencing the issue at least a few times a year, $79\,\%$ rated it a moderate or serious problem. Eck et al. pointed out the importance of replications to corroborate their findings and the need for further research assessing the role played by additional factors like project and organizational domains. We are not only following this call but go beyond existing experience reports in both number of participants and detail of questioning.

B. Root Causes

Luo et al. [9] first investigated flaky tests by inspecting 201 commits that likely fix flaky tests in 51 open-source Apache projects. They identified ten possible root causes for flakiness, of which concurrency, especially asynchronous waiting, was the most prominent one. Later studies confirmed and extended this list by inspecting test-related bug reports [10], commits relevant to flakiness in Android projects [11], previously fixed flaky tests extracted from the Mozilla bug tracker [8], or pull requests fixing flaky tests in Microsoft projects [12]. The main limitation of these studies is that they identify and quantify root causes only by investigating fixes of flakiness, but the number of issues or pull requests concerning flaky tests might not correspond to how often developers have to deal with a specific type of flakiness. This concern is supported by the fact that the one study taking a different approach by identifying flaky tests through reruns and classifying them by inspecting their code [4] disagrees with previous studies finding networking and randomness to be the most prevalent causes. With our survey, we search for the causes of flakiness that are most prominent from the developers' point of view.

C. Consequences

Trying to quantify the impact of flaky tests, Rahman and Rigby [13] showed that Firefox builds containing failing flaky tests receive more crash reports than average builds. Other studies showed that even if flaky tests are rare, they can still cause many build failures [12], [14], [15] and that up to $16\,\%$ of computational resources are spent rerunning flaky tests [16]. Such metrics can serve as proxies to measure the impact of test flakiness, although its most commonly stated effects, namely *losing trust in testing* and *wasting developer time*, have only rarely been studied [8]. In our survey, we compare different forms of resource waste, trust loss, and other impediments.

The loss of trust in tests has been reported to be asymmetric [2]: Passing tests are seen as an indication for the absence of regression, whereas failures are seen as a hint to run the test again. This *idiosyncrasy of software development* [2] is an assumption often underlying flakiness-detection techniques. While hints for this phenomenon can often be found in the literature (e.g., "You'll need to run a test suite multiple times to determine whether you have a bug or a flaky test" [17]), these are only individual examples that indicate the presence of such an asymmetric perception. Using statistical tests, we aim to scientifically validate the existence of this phenomenon.

D. Mitigation Strategies

Similar to bugs, flakiness is considered a problem that cannot be eradicated completely [18]. For example, despite Google's efforts to reduce test flakiness, they are facing a constant rate of $1.5\,\%$ of all test executions producing flaky results [6]. To mitigate the problem, multiple approaches have been proposed: 1) **Rerun & Disable:** This is the simplest way to deal with flakiness, but it wastes resources and leads to delay or loss of test feedback. 2) **Flag:** Automatically rerun tests via annotations [19]–[21]. 3) **Auto Detect:** Minimize the number of reruns by inspecting version control histories (DeFlaker) [22], smart test shuffling (iDFlakies) [3], identifying test dependencies (PraDet) [23], or predicting test flakiness via static [24], [25], or hybrid [26] analysis. 4) **Auto Debug:** Find root causes by comparing runtime parameters [14] or coverage behavior [27], and propose fixes for order-dependent tests [28] or asynchronous waiting [12]. 5) **Incentives & Penalties:** Strong management oversight, assign team members and time slots to fix flaky tests [29]. 6) **Quantify & Visualize:** Create attention for the problem [19], outline expected reward for fixing [2], and use dashboards to visualize flakiness [17], [29]–[31]. 7) **Auto report & disable:** Submit bug reports and move tests to separate test suite (*Quarantining*) [6], [12], [19], [32]. To the best of our knowledge, there is no study on how frequently these techniques are being applied. We aim to close this gap in order to provide researchers valuable feedback on the adoption rates of their approaches.

E. Wishes

So far, little is known about the developers' desires regarding what imaginable defense mechanisms against test flakiness they would like to possess. In their survey, Eck et al. [8] provided participants with a list of eight literature-selected pieces of information and asked which of those are most important and most difficult to obtain in order to fix a flaky test and what additional challenges they face. They found that reproducing the context leading to the test failure and understanding the nature of the flakiness are the most important and challenging needs and that designing test code properly to avoid flakiness is an additional challenge. Unlike them, we ask developers a more open question, hoping to collect creative ideas that are not restricted by existing literature.

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

III. Study Setup

Through our study, we aim to empirically answer the following research questions:

RQ1: How prevalent and severe is the problem of flakiness?
RQ2: What are the causes of flakiness?
RQ3: What are the consequences of flakiness?
RQ4: What are common strategies to mitigate flakiness?
RQ5: What tools or information would developers wish for to better handle flaky tests?

A. Target Audience and Participant Selection

Our target audience are professional software developers and testers since they experience potentially existing flakiness in its full extent and unfiltered form. Picking a sampling strategy for this population is challenging since one has to balance two trade-offs: Limiting the population to developers working for only one (or a few) companies paints a detailed picture and controls for company-specific variables like policies or organizational structures, which all participants have in common. However, the resulting findings might not generalize beyond that company. On the other hand, broadening the scope by sampling participants from different companies, countries, and industries creates a more general overview, although the results might be influenced by unknown variables. By conducting our study on two different sample populations, one from a specific company and one global, we hope to address these concerns and therefore strengthen both internal and external validity of our survey. In the following, we refer to these two groups as *automotive* and *global* participants.

1) Automotive Survey: All automotive participants work with the BMW group. To promote our survey, we contacted teams that match our target audience and reached out to focus groups where we presented our planned work. We encouraged them to participate by pointing out the opportunities to learn more about known root causes of test flakiness and to see how other developers and researchers deal with the problem. Apart from that, no monetary or other incentives were given.

2) Global Survey: To retrieve a global sample of our target audience, we used Prolific [33], an online service for recruiting subjects for scientific experiments. We employed a multi-stage filtering process to ensure that the participants match our target audience: First, we filtered for full-time and part-time employed individuals stating that they have programming experience and are proficient in at least one software development technique from Prolific's 17-element-long list. Furthermore, we only considered participants with a 100 % approval rating, meaning that they participated honestly in multiple other studies. These pre-defined options provided by Prolific resulted in a sample of about 7 200 out of 260 000 recently active Prolific members.

Since we are specifically looking for professional software programmers and testers, and Prolific does not provide a suitable filtering option for the participants' occupation, we conducted a prescreening study recruiting 604 individuals who matched the above demographic options, asking them if they "professionally develop or test software", "currently work on a software project", and "write code or tests in their project". Like all Prolific studies, the survey was simultaneously advertised to all members matching the demographic filter until the desired number of submissions had been reached, meaning that participants were sampled on a first-come, first-serve principle. Of the 302 prescreening study participants who answered all three filter questions in the affirmative, 233 participated in our main survey. Participants received an above-average financial reward (based on Prolific's recommendation).

Table I: Structure of the questionnaire.

Question		Type (* allows additional free text)	Answering Options (× Statements)
Demographic			
1.	Platforms	multiple-choice*	12
2.	Software Types	multiple-choice*	20
3.	Code Size	single-choice scale	6
4.	Team Size	single-choice scale	6
5.	Test Levels	multiple-choice*	7
6.	Testing Practices	multiple-choice*	8
Experience with Flakiness			
7.	Prevalence	single-choice scale	6
8.	Severity	single-choice scale	4
9.	Root Causes	Likert scale*	5 × 21
10.	Consequences	Likert scale*	5 × 6
11.	Mitigation Strategies	Likert scale*	5 × 12
12.	Wishes	free text	

B. Survey Design

The survey[1] starts with a short introduction to the issue of test flakiness. To establish a common understanding of what flaky tests are, it defines them as "*Tests that behave non-deterministically, so they sometimes pass, sometimes fail, even though there were no changes in the code they test*", followed by two examples taken from a previous study [4].

The questionnaire consists of 12 questions, of which the first six are demographic questions, and the other six are related to the participant's view on flakiness (Table I). The latter are directly linked to the research questions. The survey allows participants to skip questions for which they have no opinion to avoid demotivating them or receiving random answers. To refine our survey, we piloted it on eight developers from outside and inside BMW, who were not included in the final survey, and revised based on their feedback.

1) Demographic Questions: We avoid asking about specific languages or frameworks as these come in an overwhelming number and follow current trends and are therefore subject to frequent changes. Instead, our survey starts with two questions asking participants about (1) the platforms they are developing for (such as *Desktop*, *Mobile*, or *IoT*) and (2) the types of software they are developing (such as *Entertainment* or *Security*). We believe that these statements offer a proper abstraction that is more stable over time while still allowing the inference of possibly used technology stacks. Both questions are taken from Jetbrains's *State of Developer Ecosystem 2020* [34] survey. Our survey provides the same answering options as

[1]https://doi.org/10.6084/m9.figshare.16727251

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

the Jetbrains study plus one that has been identified while piloting the questionnaire (software type *Education*), as well as automotive-specific options: The platforms *Classic* and *Adaptive* AUTOSAR[2] are the two most common architectures of electronic control units (ECUs) including software running in cars. The Classic platform is used for computationally less demanding purposes like climate control but also for safety-relevant tasks such as airbag release and stability control systems due to its real-time computing capabilities. The Adaptive platform has more computing power and is, among others, used to implement automated driving functionalities. Furthermore, our survey offers three automotive-specific types of software: *Chassis* (e.g., anti-lock breaks), *Drive Train* (e.g., engine control), *Automated Driving and Driver Assistance* (e.g., cruise control and lane-keeping); and adds *Infotainment* (e.g., navigation) to the existing category *Entertainment*.

To estimate the size of our participants' operations, they are asked (3) to specify the number of lines of source code of their main product or project as well as (4) how many developers are working on it. Both are single-choice questions where the answering options follow a logarithmic scale. Lastly, our survey asks about (5) the usage of different kinds of tests like *unit* or *integration tests* and (6) testing practices such as *regression testing* or *fuzzing*. The answering options consist of commonly established testing levels and practices as well as more automotive-specific choices like *hardware-in-the-loop tests*. For multiple-choice questions, participants can add further options using free text.

2) RQ1: Prevalence & Severity: Questions (7) and (8) ask the developers how often they experience flaky failures and how problematic they perceive them. The prevalence should be specified on a 6-point scale ranging from *never* to *multiple times a day*. To measure the severity of flakiness, participants should state if it poses to them a *non-existing*, *minor*, *moderate*, or *serious problem*. These scales match the ones used by Eck et al. [8], allowing us to compare our findings to theirs.

3) RQ2: Root Causes: To investigate where flakiness comes from, the survey presents the participants (9) with 21 statements describing different root causes of flaky test behavior and asks them to specify how often they experience each on a qualitative 5-point Likert scale (*never* to *always*), following the template by Kasunic [35]. They can also add options to the list via a free text field.

We derived the statements from 15 root causes identified by prior studies [4], [8], [9], splitting *networking* into local bad socket management and remote connection failure [9], and *order dependency* into victims and brittles [28]. These distinctions make our survey both more precise and more easily understandable for the participants. Furthermore, we added three root causes, which were mentioned to us while piloting the survey, namely reading values from *uninitialized variables*, *differences between compilers* (like undefined behavior or optimizations), and the usage of *random, non-seeded values for testing*. In addition, participants can also indicate that they do not know what causes their tests to become flaky. To ensure that the statements are easily understandable, each of them is formulated in a concise sentence, e.g., "Prior test did not clean up properly after running" representing the root cause *order dependent victim*.

4) RQ3: Consequences: Using another matrix of Likert scale questions (10), the survey addresses the consequences of flakiness by presenting the participants with six statements describing possible effects of having flaky tests in their test suite, that we derived from literature. First, developers are asked how often they see 1) failing tests being rerun without being analyzed or 2) passing tests being rerun suspecting hidden bugs. These two statements investigate the loss of trust in testing. Second, they are asked how often 3) time or 4) computational resources have to be spent for rerunning, debugging, or fixing flaky tests. Lastly, the developers should rate how often 5) pull requests cannot be merged and 6) releases are delayed due to test flakiness. Apart from rating the prepared statements, participants are encouraged to mention further consequences via a free text field.

5) RQ4: Mitigation Strategies: Since we do not expect the participants to be familiar with specific tools, they have to (11) rate their usage of the different types of mitigation strategies outlined in Section II-D on a Likert scale and specify any other strategies they apply via free text options.

6) RQ5: Wishes: In the last question, participants are asked (12) what information or tools would be most helpful to them in order to better handle flaky tests. Unlike the other questions, there are no pre-defined answers, only a free text field. Through this question, we hope to identify possible directions for future research in order for it to generate practical value.

C. Processing Answers

1) Consolidating Likert Scale Answers: In order to concisely depict the responses to each Likert scale statement in the limited space of this paper, we transformed the data into integer numbers and calculated their mean using the following mapping: Never $\rightarrow$ 0, Rarely $\rightarrow$ 1, Sometimes $\rightarrow$ 2, Very Often $\rightarrow$ 3, Always $\rightarrow$ 4. This allowed us to compress the answers for each statement to a single number.

2) Coding Free Text Answers: For multiple-choice and Likert scale questions our survey allows additional free text answers. Another source of free text answers is the last question on the developers' wishes. Following existing guidelines for conducting surveys in software engineering [36], we coded these responses using a quantitative content analysis: As an initial set of codes, we used the pre-defined answering options; for the wishes question we used the categories of the mitigation strategies. If necessary, we modified this schema by adding new codes, in which case we re-applied the new schema to all data already coded. The coding was individually executed by both the first author and another software testing researcher. In case of disagreements, which appeared in 28 out of 265 cases, these were resolved via an in-depth discussion. Note that one response can have multiple codes assigned to it since some mention multiple different aspects.

[2]Automotive Open System Architecture (https://www.autosar.org/)

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

D. Relating Answers

To measure the impact of demographic variables on how developers experience flakiness, we used ordinal logistic regression [37]: We trained a model to predict each response that the participants gave about their experience with flakiness (like prevalence, severity, or any Likert scale statement), based on their answers to the demographic questions. For each demographic variable, we then looked at its coefficient and confidence interval to see if it constitutes a good predictor. We considered a connection to be significant if the $95\,\%$ confidence interval does not cross the line of no effect, which is zero for the coefficient. To measure the strength and direction of a connection, we calculated its odds ratio (OR), which is the exponentiated coefficient. An odds ratio of 1.5 between the variable *platform: Desktop* and *prevalence* would for example mean, that for participants developing desktop applications, the odds of being more likely to experience flaky failures are 1.5 times that of participants not developing desktop applications, holding constant all other variables. An odds ratio above one indicates a positive relation, whereas an odds ratio below one indicates a negative influence on the target variable.

A possible issue with this method is that demographic variables might be correlated, leading to multicollinearity, which reduces the precision of the estimated coefficients and their significance [38]. To test our data for multicollinearity, we calculated the variance inflation factor (VIF) for each demographic variable and removed variables with elevated values. This was the case for the size of the participants' code base and the size of their teams. Excluding these variables also allowed us to use our entire dataset of 335 responses since three participants did not answer these two questions.

E. Threats to Validity

1) External Validity: The 233 global participants hired via Prolific [33] showed typical demographic attributes for the target population (Section IV-B), however, the mere fact that a developer is active on Prolific might itself carry a bias. Similarly, for the 102 automotive participants, their answers might not be representative of the entirety of automotive software engineers. Furthermore, as we found significant differences between developers working in various domains, their combined results will be influenced by the distribution of the regarded population. For this reason, we conducted our survey on two different sample populations and report their results separately.

2) Construct Validity: To select suitable individuals for our global survey, we employed a multi-stage filtering process. However, since all this information is self-provided and cannot be verified, our selection process is susceptible to false declarations. Indeed, we excluded three participants who filled out the prescreening study multiple times, giving different responses. To recruit automotive participants, we used convenience sampling, which might suffer from self-selection or voluntary response bias: Developers who experience flakiness more often might be more likely to respond to our survey than those who never heard of it. Therefore, their numbers might be elevated compared to a random sample.

3) Internal Validity: Seven out of our 12 questions use multiple-choice answers or statements that should be ranked. We derived possible answering options from reviewing both white and grey literature and piloting the questionnaire. While we might have been influenced by our subjective judgement about what constitutes a worthy answering option, this risk is mitigated by the possibility to provide free text answers, which were coded independently by two researchers. This process might however be influenced by our interpretation of the responses.

IV. Results

A. Number and Quality of Responses

In total, we collected 335 at least partially complete submissions, 102 from automotive participants and 233 from global participants. 301 of these are fully complete, meaning that all non-free-text questions have been answered. For all further analysis, we consider all 335.

B. Demographics

Inspecting the responses to the demographic questions of our survey, we see a clear difference between automotive and global participants (Table II). Global participants mainly develop websites, databases, and utilities for desktop-, web-, and mobile systems, strongly matching the Jetbrains survey [34] from where these questions were taken. Their projects typically encapsulate not more than 100k lines of code and 5 people, which is again similar to the Jetbrains study ($52\,\%$ work in project teams of 2-7 people) and Eck's survey [8] ($76\,\%$ work in teams of up to 10 people). On the other hand, while also working on utilities and libraries, automotive participants develop software for automated driving, IT infrastructure, and infotainment for IoT-, desktop-, and AUTOSAR platforms. Their operations are larger, comprising more than 100k lines of code and over 20 people on average. Furthermore, automotive participants make far greater use of automated testing, continuous integration, regression testing, and parametrized testing than global participants.

Through Prolific, we also have access to other demographic data about the global participants, including age, sex, student status, and various nationality and language variables. We did not record any such data for the automotive survey because we do not use it to answer any of the RQs. However, for the global survey, this data can help us to verify if we achieved our goal of retrieving a global sample of software developers and testers. In general, participants are rather young, with a majority being under 30 (median 28, youngest 18, oldest 63), which is also reflected in the high proportion of students ($35\,\%$). This is comparable to Jetbrains's *The State of Developer Ecosystem 2020* [34], where $24\,\%$ of all participants were (working) students and $63\,\%$ were younger than 30. In terms of nationality, we see a clear tendency towards the western hemisphere, with $69\,\%$ coming from Europe and $17\,\%$ from North America, which is similar to another Prolific-based software engineering survey [39] but slightly different from the Jetbrains study, which also features many participants from China and India.

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

Table II: Answers to demographic questions. Color maps are proportional to values and question specific.

Platform (PL)	Global	Auto	Both
Desktop	55 %	40 %	50 %
Mobile	37 %	15 %	30 %
Web (Back-end)	47 %	22 %	39 %
Web (Front-end)	53 %	15 %	41 %
WebAssembly	3 %	2 %	2 %
Server / Infrastructure	19 %	26 %	21 %
IoT / Embedded	10 %	46 %	21 %
Classic AUTOSAR ECU (CAS)	1 %	19 %	6 %
Adaptive AUTOSAR ECU (AAS)	0 %	31 %	10 %
Consoles (Xbox / PlayStation / Nintendo etc.)	3 %	1 %	3 %
Not decided yet (research / proof of concept)	3 %	3 %	3 %
I don't develop anything	1 %	2 %	1 %
Other	2 %	10 %	4 %
Software Type (ST)			
Automotive Chassis	1 %	7 %	3 %
Automotive Drive Train	0 %	2 %	1 %
Automated Driving, Driver Assistance	1 %	37 %	12 %
Augmented Reality / Virtual Reality	3 %	6 %	4 %
Business Intell. / Data Science / Machine Learn.	17 %	13 %	16 %
Blockchain	4 %	0 %	3 %
Database / Data Storage	34 %	11 %	27 %
Education / Training	15 %	2 %	11 %
Entertainment / Infotainment	12 %	24 %	16 %
Fintech (Finance)	9 %	0 %	7 %
Games	18 %	5 %	14 %
Hardware	12 %	7 %	10 %
Home Automation	6 %	4 %	5 %
IT Infrastructure	17 %	24 %	19 %
Libraries / Frameworks	8 %	26 %	14 %
Programming Tools	12 %	20 %	15 %
Security	9 %	6 %	8 %
System Software (e.g. OS driver)	6 %	12 %	8 %
Utilities (small apps for small tasks)	29 %	26 %	28 %
Websites	39 %	11 %	31 %
Other	6 %	11 %	7 %
Code Size			
< 1k	12 %	1 %	9 %
1k - 10k	32 %	8 %	25 %
10k - 100k	33 %	25 %	30 %
100k - 1M	15 %	32 %	21 %
1M - 10M	5 %	17 %	8 %
> 10M	3 %	15 %	7 %
no answer	0 %	2 %	1 %
Team Size			
Just me	23 %	5 %	18 %
2 - 5	45 %	23 %	38 %
6 - 10	15 %	7 %	13 %
11 - 20	8 %	16 %	10 %
21 - 100	7 %	13 %	9 %
> 100	2 %	36 %	13 %
no answer	0 %	1 %	0 %
Test Level (TL)			
Unit Tests	61 %	94 %	71 %
Component Tests	27 %	58 %	37 %
Integration Tests	52 %	83 %	61 %
System Tests	39 %	51 %	43 %
Software in the loop Tests (SiL)	15 %	26 %	18 %
Platform in the loop Tests (PiL)	8 %	11 %	9 %
Hardware in the loop Tests (HiL)	5 %	19 %	9 %
Other	5 %	3 %	4 %
Testing Practices (TP)			
Manual Testing	85 %	64 %	79 %
Automated Testing	61 %	88 %	70 %
Continuous Integration	33 %	96 %	53 %
Regression Testing	25 %	60 %	36 %
Fuzzing	4 %	12 %	7 %
Property-based Testing	6 %	2 %	4 %
Parameterized Tests	13 %	51 %	24 %
Test Generation	7 %	13 %	9 %
Other	0 %	2 %	1 %

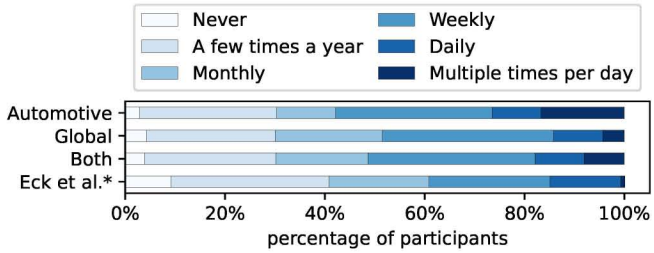


Figure 1: How often do you experience flaky failures?
* Eck et al. asked "How often do you deal with flaky tests?"

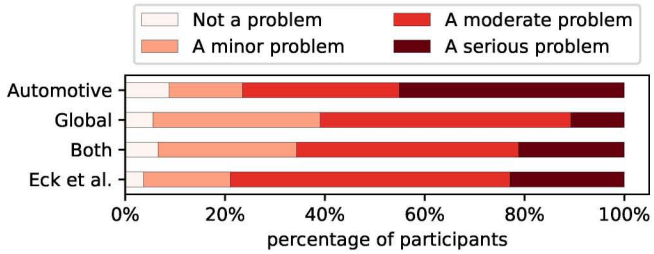


Figure 2: How problematic is flakiness for you?

Of our global participants, 81.1 % are male, which almost exactly matches the distribution from Russo et al.'s survey [39] (81.4 %), who argue that this is a comparably high female ratio for the field (93 % of participants in Jetbrains's 2021 study were male[3]). Overall, we see no strong bias towards any specific groups but rather a strong overlap with other representative studies, indicating that we retrieved a diverse, global sample of software developers and testers.

C. RQ1: Prevalence & Severity

Figures 1 and 2 show how frequently participants experience flaky failures and how problematic they perceive flakiness. Almost every participant has to deal with flaky failures at least a few times a year (only 13 never experience the problem), a majority experiences the issue at least weekly, and 66 % rate it a moderate or serious issue!

We also see a considerable difference between our two participant groups as automotive participants seem to be more affected by flakiness. Consulting the odds ratios of the demographic variables (Fig. 3) offers an explanation for this observation: The usage of automated testing and continuous integration (bottom two bars) are both strong positive predictors for the prevalence (and partially severity) of flaky failures, and both testing practices are used by automotive participants at a far higher rate (Table II). This confirms other studies reporting flaky failures to be particularly common in continuous integration environments [14], [15] and explains why test flakiness has been an uprising issue for the last decade, where continuous integration has received wide adoption[4].

[3]this question was not asked in the 2020 survey.
[4]both Jenkins and Travis CI celebrate their 10th birthday this year

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

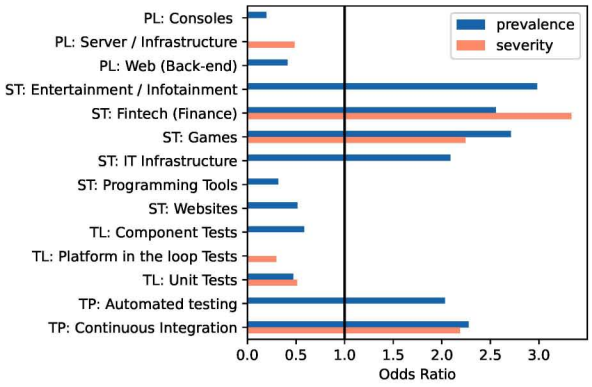


Figure 3: Demographic variables with significant impact on prevalence and/or severity of flakiness. The black bar separates negative influence (< 1) from positive influence (> 1).

Table III: Means per platform, sorted by prevalence. Color-scales are column-specific. Only showing platforms with at least 10 participants. Rows overlap (share participants) since the platform question allows multiple choice.

Platform	Prevalence	Severity	Code Size	Team Size
min, max, mean	0, 5, 2.43	0, 3, 1.80	0, 5, 2.15	0, 5, 1.92
Web (Back-end)	2.13	1.66	2.07	1.55
Web (Front-end)	2.19	1.67	2.05	1.53
Classic AUTOSAR ECU	2.33	2.19	3.29	3.95
Server / Infrastructure	2.47	1.72	2.17	2.10
Desktop	2.48	1.74	2.03	1.72
Mobile	2.50	1.70	1.85	1.61
IoT / Embedded	2.67	2.10	2.74	2.81
Adaptive AUTOSAR ECU	2.88	2.48	3.62	4.39

Figure 3 also shows several unexpected relations: Participants developing websites, particularly back-end applications, seem to experience flaky failures less often despite using continuous integration and automated testing to the same or higher degrees as other participants. While this offers an additional explanation for why automotive participants are more strongly affected, we would have expected quite the opposite, since one of the most common causes of test flakiness (asynchronous waiting) largely refers to front-end–back-end communication. Other surprising observations include the strong negative effect of platform *Consoles* and the strong positive effect of software types *Entertainment / Infotainment*, *Fintech*, and *Games*. However, it has to be noted that these are the least distinctive variables, as only a few participants fall into these groups (min 3 %, max. 16 %), particularly compared to continuous integration, which 53 % of all participants specified, making them more susceptible to the answers of individual participants.

Lastly, we look at the correlation between the two variables prevalence and severity, which is both strong, positive, and significant ($\tau = 0.39$, $p < 0.001$). While we cover the implications of this observation in the context of RQ3 (Section IV-E), there are two noteworthy exceptions (Table III): Developers working on Classic AUTOSAR projects experience flaky failures less often than average while rating flakiness a more serious problem than most other projects. This observation can be explained by the type of applications running on this platform: The Classic platform is used for safety-relevant real-time software, such as airbag release or stability control systems. As a result, these programs have strict requirements towards deterministic behavior, explaining both the rarity and the criticality of flaky failures. Furthermore, correcting errors in the field can be very costly, especially since software failures might also damage hardware parts.

Table IV: Mean rating per root cause from all participants, ranging from never (0) to always (4). For the demographic influences, we excluded groups with less than 10 participants.

Root Cause	Both	Demographic influences (strongest 3)
Concurrency	1.80	⊖ Research/PoC, ⊖ CAS, ⊕ Cont. Integr.
Order-dependency (victim)	1.74	⊕ Home autom., ⊕ Database
Async Wait	1.70	⊕ Test generation, ⊖ System Test
Unknown Cause	1.63	⊖ Test gen., ⊕ Fintech, ⊕ Entertainm.
Network (remote)	1.60	⊖ Chassis, ⊕ HiL Test, ⊕ Games
Test case timeout	1.56	⊕ Games, ⊕ System SW, ⊕ Fintech
Test suite timeout	1.51	⊖ Prop. based Test, ⊕ System SW, ⊕ AR
Order-dependency (brittle)	1.48	⊕ Database, ⊖ System Test
Infrastructure	1.40	⊕ Research/PoC, ⊕ Games
Uninitialized variables	1.26	⊖ Research/PoC, ⊕ Hardware
Time	1.25	⊖ CAS, ⊕ HiL Test, ⊕ Fintech
Randomness	1.19	⊖ Automat. Driving, ⊕ Games, ⊕ Edu.
Floating point	1.16	⊕ Games, ⊕ Fintech, ⊕ PiL Test
Too restrictive range	1.13	⊖ CAS, ⊕ Fintech, ⊕ Games
Resource Leaks	1.09	⊕ Games, ⊖ Prog. Tools, ⊕ Sever/Infr.
Compiler differences	1.09	⊕ Fintech, ⊕ Fuzzing, ⊕ Hardware
Unordered collection	1.06	⊕ Hardware, ⊕ Education
Platform dependency	1.06	⊕ Fuzzing, ⊕ PiL Test, ⊕ Fintech
Non-Seeded Testing	0.96	⊕ Fuzzing, ⊕ Hardware, ⊕ Comp. Test
Network (local)	0.93	⊕ Games, ⊕ Hardware, ⊖ Manual Test
IO (garbage collector)	0.86	⊕ Fintech, ⊕ Fuzzing, ⊕ PiL Test

Participants developing mobile applications show rather the opposite picture: They experience flaky failures frequently while rating it a less serious issue. Talking to multiple representatives of this group, we found that this is caused by Android emulators and testing tools, which are plagued by non-determinism. One participant specifically mentions that they do not wish for any additional tools to tackle flakiness but just for more stable Android testing tools since about 80 % of their flakiness originates from the test environment, not the tests themselves. This observation is confirmed by Listfield [5], who reports that 25 % of their tests using the Android emulator are flaky (compared to 1.7 % of all tests) and that the Android Emulator is the only tool for which the increased amount of flakiness cannot be explained by tests simply being larger.

Summary ***(RQ1: Prevalence and Severity of Flakiness)*** Test flakiness is a common and severe issue, especially among those using continuous integration. Domain-specific factors like flaky emulators or safety relevance influence the prevalence and severity of flaky failures.

D. RQ2: Causes

Table IV depicts our results regarding the causes of flakiness. In contrast to the previous question, there is strong agreement between global and automotive participants: Both rank concurrency as the most prevalent cause of flakiness and order-dependent victims, asynchronous waiting, and unknown causes among the top five. For each type of root cause, Table IV

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

Table V: Mean rating per consequence; never (0) - always (4).

Consequences	Global	Auto	Both
Wasting developer time	2.32	2.73	2.44
Rerun failures without analyzing	1.89	2.54	2.08
Merging pull requests is harder	1.73	2.89	2.08
Wasting computational resources	1.65	2.54	1.91
Rerun passes suspecting hidden bugs	2.00	1.54	1.86
Delayed releases	1.73	1.86	1.77

lists the three most important demographic factors determined using logistic regression, denoting whether the factor leads to higher or lower prevalence. We observe multiple phenomena of which we highlight three trends we found most distinctive:

- First, concurrency-related issues are less common in projects that have not decided on a target platform, such as research projects or proofs of concept. This might be the case because such software has little use for concurrency, as it is neither relevant for performance optimization through parallelization nor for multi-user interaction. Developers targeting Classic AUTOSAR ECUs—being quite the opposite of a proof of concept—also experience fewer issues with concurrency, however, this is the case because concurrent processes are meticulously controlled using time slices, avoiding unforeseen behavior (see also root cause *time*).
- Second, we see that developers working on databases have issues with order-dependencies, which is intuitive since these are main carriers of global states that have to be properly reset after every test execution.
- Lastly, the two new categories *Uninitialized variables* and *Compiler differences*, which were pointed out to us by automotive developers during the survey pilot, are experienced by hardware developers and are also common with global participants. This is worth mentioning, especially since these causes only exist in certain languages.

We also received 27 free text answers, largely from automotive participants (20x), who mention infrastructure issues (9x), mostly concerning CI (6x), for example, failing to pull docker images, as well as flaky behavior of the device under test (4x). Both are indicative of the type of software they develop, which is largely embedded programs requiring specialized hardware setups. These may result in difficult to control dependencies between hardware and environment [40]. Global participants frequently mention old code and undocumented changes (7x), for example, to remote APIs, causing tests to be flaky.

Summary *(RQ2: Root Causes of Flakiness)* While causes vary for different kinds of software and tests, concurrency and order-dependencies are the most common causes of test flakiness. Uninitialized variables, differences between compilers, and unexpected API behavior are further previously unmentioned but not uncommon reasons for flaky tests. However, in many cases, the root causes remain unknown.

E. RQ3: Consequences

Table V shows the average ratings to the six statements from question nine, "Which negative effects of flaky tests have you experienced?" Wasting developer time is perceived as the most severe consequence of test flakiness. Nevertheless, other impediments are also present in everyday business: For each statement, at least half of all participants stated that they experience it at least sometimes. Across the board, automotive participants experience the negative impact of test flakiness more often than global ones, an effect that we already observed in RQ1. The only exception is the option *losing trust in passing tests to indicate the absence of regression* (5th row): While global participants claim to suffer very similar amounts of trust loss towards both failing and passing test cases, automotive participants retain confidence in passing tests at a far higher rate while mistrusting failures much more frequently. This might be caused by global developers using more manual testing, where instinct plays a large role in finding hidden bugs. Both participant groups agree that developer time is wasted more frequently than computational resources.

When submitting code changes, automotive participants are specifically affected, with $76\,\%$ stating that flaky tests hinder them in merging pull requests very often or always. A likely explanation for this observation can be found in Table II, where we notice that almost all automotive participants use continuous integration, while only $33\,\%$ of global developers do so. Additionally, more than $60\,\%$ of all participants see releases getting delayed at least sometimes due to test flakiness.

We also received 31 free text answers listing other consequences of test flakiness. Many comments fall into the categories *wasting developer time* (8x) and *losing trust in testing* (7x), which confirms the selection of our pre-defined statements. Both automotive and global participants also repeatedly mention frustration (5x) as a consequence of test flakiness. Further responses state that flakiness results in production failures (4x), creates pressure by blocking or requiring last time changes (2x), and causes uncertainty if test failures actually indicate regressions (2x). Interestingly, deactivating tests and writing rerun bots, which we consider mitigation strategies, were mentioned as negative consequences of flaky tests, indicating that the developers are dissatisfied with the way flaky tests are currently being handled.

To verify the existence of an *asymmetric perception* [2] of developers towards test executions, we looked at two aspects: First, we checked if the failure rate of a flaky test has influence on how severely developers perceive it. To measure this effect, we calculated the correlation between the two ordinal variables prevalence (question 7) and severity (question 8), using Kendall's tau. We found a strong, positive, and significant connection between those variables ($\tau = 0.39$, $p < 0.001$), which confirms that flaky tests with high failure rates are also perceived as more problematic.

Second, we checked if developers lose trust in failing tests at the same rate as they do in passing tests. To measure this effect, we verified if the responses of the two statements *Rerun passing*

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

Table VI: Mean rating per mitigation strategy; never (0) - always (4).

Mitigation strategies	Global	Auto	Both
Rerun	2.70	3.05	2.80
Rerun in different environment	2.10	1.37	1.89
Auto report	1.66	1.12	1.51
Flag	1.53	1.45	1.51
Shuffle test order	1.59	1.00	1.43
Incentives & Penalties	1.39	1.40	1.40
Disable	1.18	1.59	1.29
Quantify	1.34	1.09	1.27
Visualize	1.34	1.04	1.26
Auto detect	1.21	1.02	1.16
Auto debug	1.34	0.53	1.11
Auto disable	1.04	1.01	1.03

tests suspecting hidden bugs and *Rerun failing tests without analyzing the failure* originate from the same distribution. Since a Shapiro-Wilk Test showed that their values are not normally distributed, we used a Wilcoxon signed-rank test, which resulted in a p-value of less than 0.001, meaning that the two statements do not originate from the same distribution. Together with the higher mean value of *Rerun failures without analyzing* (Table V), this shows that developers lose trust in the signaling power of test failures more than in the signaling power of passing tests. However, there is a large difference between the two groups: While this effect is extremely strong for automotive participants, it is not significant when looking only at global participants ($p = 0.13$). This difference might be caused by the fact that automotive participants experience flaky failures more often, which is reported to make developers more likely to ignore genuine test failures [41], or—as we suspect—rerun them. Furthermore, only global participants mention production failures as a consequence of test flakiness, an experience that might lead them to also rerun passing tests.

Overall, we can confirm the existence of an asymmetric perception, showing that many developers look at test executions differently depending on the test's outcome. A possible explanation for this phenomenon could be that most flakiness originates from the test code ($\sim 90\,\%$ according to [8]) or the test environment, so most flaky tests are indeed false alarms and not hidden bugs. However, while being rarer, flakiness caused by hidden bugs is arguably more devastating. Considering flaky tests solemnly as false alarms is therefore a dangerous action, which cannot be recommended. The asymmetric perception can therefore be seen as an indicator of the deceptive power flaky tests possess in masking actual bugs [27].

Summary *(RQ3: Negative Effects of Flaky Tests)* Flakiness destroys trust in testing and causes frustration. The degree of this also depends on the test outcome. Flaky tests inhibit project development by wasting developer time more than computational resources and blocking pull requests.

F. RQ4: Mitigation Strategies

Table VI shows the responses to the second last question of our survey, touching the mitigation strategies developers currently apply to deal with flakiness. Rerunning tests is by far the most common reaction, while automated techniques rank lowest. This might be the case because many sophisticated mitigation tools are implemented as plugins to Java build tools [3], [22], [26], however, many industrial projects—especially in the automotive industry—do not use Java, preventing developers from applying these techniques without re-implementing them.

Furthermore, we received 20 free text answers and 6 responses to the last question (wishes), which also report currently applied strategies. The majority of these describe rewriting the test case to remove its flakiness (11x), manually investigating the root cause (7x), and rerunning tests (5x). Two developers see communicating with colleagues as a good strategy, namely talking to the test owner and raising awareness for memory errors. One participant mentions that using Sonarlint [42], an IDE extension that identifies and helps to fix quality and security issues, is useful in preventing more obvious cases of flakiness.

Summary *(RQ4: Currently Applied Mitigation Strategies)* Rerunning and rewriting test cases are by far the most dominant approaches to deal with flaky tests, while automated techniques are only rarely used.

Table VII: Wishes for tools or information to better handle test flakiness expressed by our participants.

Wish	Count	Wish	Count
Visualization	32	Rerun	17
test result history	14	shuffle execution order	4
which tests are flaky	4	in different environment	3
program behavior	3	Manual debugging tools	16
Auto Detection	31	Stable infrastructure	16
via static analysis	12	machine readable infrastructure status	5
Auto Debug	28	Logging	13
find root cause	6	Management	11
find location	5	priority + resources	4
find failure cause	4	good team	4
Education	25	incentives & penalties	2
guides, examples, best practices	16	issue bug reports	2
help from colleagues	3	Reproducibility	9
training	2	failure replay	3

G. RQ5: Wishes

We received 187 suggestions and wishes for tools and information that developers would like to have in order to better deal with flaky tests. While automotive participants answered this question less often than their global counterparts (23 / 102 vs. 164 / 233), their answers are more detailed (median 156 characters vs. 67 for global participants). After coding the submissions, we discarded 30 responses from global participants because they were not interpretable (14x) or very generic like "Don't know" (7x), "Nothing" (5x), or "Anything" (3x). Furthermore, we found four answers from global participants that do not mention any wishes but instead describe root causes or mitigation strategies. We ignored these here but referred them to the respective RQ. Table VII summarizes the 153 meaningful, on-topic answers.

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

We find a strong desire for better visualization of flakiness, coming from both global participants (21x) as well as half of all the responses of automotive participants (11x)! They specifically ask for tools to display the test result history, stating that "We run tests so often, I often miss the bigger picture" (P 187), wishing for "A tool to visualize how tests fail and succeed over time" (P 12). Such a tool should also include meta information like "on which device / environment [the test was executed]" (P 292).

The second most commonly stated wish asks for tools that can automatically detect flaky tests. Some respondents also go into details on how this should be implemented, stating that they would prefer static analysis-driven approaches, preferably an IDE plugin that can identify potentially flaky tests before executing them, similar to Findbugs [43]. Many participants also want tools that can automatically debug flaky tests to determine the flakiness's root cause or location in the code.

Various responses ask for more educational opportunities, specifically best practices and guides on how to avoid or fix flakiness, examples for flaky tests, and more support from forums like Stack Overflow. Two participants would like to receive classes on test flakiness or suggest integrating the topic into programming courses. Others mention that "no tool can replace experience" (P 65) and wish for more support from (senior) colleagues. One participant stated that they found our survey itself educational and plan to follow up on the topic.

The need for proper project management is raised by 11 participants, who ask for more (human) resources, skilled and loyal team members, and reporting of flaky tests with management overseeing the fixing progress. One participant brought up the broken window theory, emphasizing the importance of preventing minor damage and conducting small repairs as fast as possible to avoid the establishment of a don't-care attitude.

Nine participants also mention the need for ways to reproduce a test's behavior, potentially through recording and replaying inputs. Only two participants wish to know about the flakiness-introducing commit, which goes along Eck et al.'s [8] results seeing this information as less important.

Summary ***(RQ5: Wishes for Information and Tools)***

1) Dashboards to visualize test outcomes over time
2) IDE plugins to detect potentially flaky tests statically
3) Automated debugging and root causing
4) Education like examples, guides and best practices

V. Related Work

The closest related work to ours is a survey by Eck et al. [8] carried out in 2019 on 121 developers, in which they found flakiness to be a frequent and severe issue, which our results corroborate. Unlike them, however, we found connections between the domain of a project and the diffusion of flakiness in it. Like ours, the participants of Eck et al. describe the main consequences of flakiness to be the waste of developer time and the decreased reliability and trustworthiness of a test suite, however, we saw no impact on resource allocation. Similar to us, Eck et al. found a need for methods to prevent the introduction of flaky tests and for defining design patterns that support the creation of deterministic tests as well as flakiness-related anti-patterns. On top of that, we saw a strong longing for better visualization techniques, such as dashboards showing the outcome of a test over time. The usefulness of such overviews has been pointed out by grey literature [17], [29]–[31], however, it has not received much attention from researchers so far to the best of our knowledge.

Independently of our survey and at the same time, two other studies queried developers about test flakiness: Ahmad et al. [44] interviewed 14 employees from five companies, two of which operate in the automotive business. Like us, they found flakiness to be overproportionally common in the automotive sector, which might be caused by a relationship between flakiness and the context (i.e., domain) in which it has been investigated. Furthermore, they also saw a strong need to prevent flakiness through guidelines and to predict it. Parry et al. [41] surveyed 170 developers from various organizations about their experience of flaky tests. Our results corroborate their findings that flakiness has a strong impact on CI, its root causes are often unknown, and that rerunning tests, as well as emotive responses like frustration, are common reactions.

Concerning the causes of flakiness, our survey confirms multiple previous studies that found concurrency to be the major cause of flakiness [8], [9], [11], [12], the majority of order-dependent tests to be victims [4], [28], and remote networking issues to be more common than local ones [9].

VI. Conclusions

Seeking answers to the question of what researchers need to know about the developers' needs, desires, and experiences regarding test flakiness to build better defense mechanisms, we conducted an empirical study on 335 professional software developers and testers. About one third of our participants originate from the automotive sector, allowing us to investigate the influence of domain-specific properties.

Flakiness is generally perceived as a prevalent and severe issue, especially within those using automated testing. Mobile apps suffer from frequent flaky failures due to flaky emulators, whereas automotive platforms used for safety-relevant applications experience flaky failures less often but rate them as highly severe. Concurrency-related issues remain the major causes of flakiness. Furthermore, we found uninitialized variables, differences between compilers, and undocumented API changes as novel reasons for tests to become flaky.

Developers currently mostly rely on rerunning and rewriting test cases, however, their wishes towards researchers and tool developers are to provide more sophisticated techniques such as IDE plugins to detect potentially flaky tests or dashboards to visualize test outcomes over time. Furthermore, they would like to receive more information and training on the topic.

To promote future research aiming to implement these needs or to replicate our work, we provide all data freely:
https://doi.org/10.6084/m9.figshare.16727251

Acknowledgements: We thank Philipp Straubinger for his support in coding free text answers.

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–74, 2021.

[2] M. Machalica, W. Chmiel, S. Swierc, and R. Sakevych, "How do you test your tests?" Dec. 2020. [Online]. Available: https://engineering.fb.com/2020/12/10/developer-tools/probabilistic-flakiness/

[3] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "Idflakies: A framework for detecting and partially classifying flaky tests," in *International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2019, pp. 312–322.

[4] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, "An empirical study of flaky tests in python," in *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2021, pp. 148–158.

[5] J. Listfield, "Where do our flaky tests come from?" Apr. 2017. [Online]. Available: https://testing.googleblog.com/2017/04/where-do-our-flaky-tests-come-from.html

[6] J. Micco, "Flaky tests at google and how we mitigate them," 2016, accessed: 2020–10–19. [Online]. Available: https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html

[7] A. M. Memon, Z. Gao, B. N. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming google-scale continuous testing," in *International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. IEEE Computer Society, 2017, pp. 233–242.

[8] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Joint Meeting of the European Software Engineering Conference and the Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2019, pp. 830–840.

[9] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *International Symposium on Foundations of Software Engineering (FSE)*. ACM, 2014, pp. 643–653.

[10] A. Vahabzadeh, A. M. Fard, and A. Mesbah, "An empirical study of bugs in test code," in *2015 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2015, pp. 101–110.

[11] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in android apps," in *International Conference on Software Maintenance and Evolution (ICSME)*. IEEE Computer Society, 2018, pp. 534–538.

[12] W. Lam, K. Muşlu, H. Sajnani, and S. Thummalapenta, "A study on the lifecycle of flaky tests," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 1471–1482.

[13] M. T. Rahman and P. C. Rigby, "The impact of failing, flaky, and high failure tests on the number of crash reports associated with firefox builds," in *Joint Meeting of the European Software Engineering Conference and the Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2018, pp. 857–862.

[14] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2019, pp. 101–111.

[15] A. Labuschagne, L. Inozemtseva, and R. Holmes, "Measuring the cost of regression testing in practice: A study of java projects using continuous integration," in *Joint Meeting of the European Software Engineering Conference and the Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2017, pp. 821–830.

[16] J. Micco, "The state of continuous integration testing@ google," 2017. [Online]. Available: https://storage.googleapis.com/pub-tools-public-publication-data/pdf/45880.pdf

[17] S. Özal, "How to deal with flaky tests," Mar. 2021. [Online]. Available: https://thenewstack.io/how-to-deal-with-flaky-tests/

[18] J. Raine, "Reducing flaky builds by 18x," Dec. 2020. [Online]. Available: https://github.blog/2020-12-16-reducing-flaky-builds-by-18x/

[19] E. Wendelin, "Preventing flaky tests from ruining your test suite." [Online]. Available: https://gradle.com/blog/prevent-flaky-tests/

[20] "flaky - a plugin for nose or pytest that automatically reruns flaky tests," accessed: 2021–09–23. [Online]. Available: https://pypi.org/project/flaky/

[21] "Flaky test handler," accessed: 2021–09–23. [Online]. Available: https://plugins.jenkins.io/flaky-test-handler/

[22] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "Deflaker: Automatically detecting flaky tests," in *International Conference on Software Engineering (ICSE)*. ACM, 2018, pp. 433–444.

[23] A. Gambi, J. Bell, and A. Zeller, "Practical test dependency detection," in *International Conference on Software Testing, Verification and Validation (ICST)*. IEEE Computer Society, 2018, pp. 1–11.

[24] A. Bertolino, E. Cruciani, B. Miranda, and R. Verdecchia, "Know your neighbor: Fast static prediction of test flakiness," 2020.

[25] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 492–502.

[26] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "Flakeflagger: Predicting flakiness without rerunning tests," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1572–1584.

[27] D. C. Celal Ziftci, "De-flake your tests: Automatically locating root causes of flaky tests in code at google," in *ICSME 2020-International Conference on Software Maintenance and Evolution*, 2020. [Online]. Available: https://www.youtube.com/watch?v=uMGWf0tFqjM

[28] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "ifixflakies: A framework for automatically fixing order-dependent flaky tests," in *Joint Meeting of the European Software Engineering Conference and the Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2019, pp. 545–555.

[29] C. Champier, "Continuous improvement of our flaky tests hunting process," Jan. 2019. [Online]. Available: https://medium.com/doctolib/continuous-improvement-of-our-flaky-hunting-process-189528a1f540

[30] S. Liviu, "A machine learning solution for detecting and mitigating flaky tests," Oct. 2019. [Online]. Available: https://medium.com/fitbit-tech-blog/a-machine-learning-solution-for-detecting-and-mitigating-flaky-tests-c5626ca7e853

[31] J. Palmer, "Test flakiness – methods for identifying and dealing with flaky tests," Nov. 2019, accessed: 2021–09–25. [Online]. Available: https://engineering.atspotify.com/2019/11/18/test-flakiness-methods-for-identifying-and-dealing-with-flaky-tests/

[32] M. Fowler, "Eradicating non-determinism in tests," Apr. 2011. [Online]. Available: https://martinfowler.com/articles/nonDeterminism.html

[33] S. Palan and C. Schitter, "Prolific. ac—a subject pool for online experiments," *Journal of Behavioral and Experimental Finance*, vol. 17, pp. 22–27, 2018.

[34] Jetbrains, "The state of developer ecosystem 2020," 2020, accessed: 2021–09–14. [Online]. Available: https://www.jetbrains.com/lp/devecosystem-2020

[35] M. Kasunic, "Designing an effective survey," Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst, Tech. Rep., 2005.

[36] J. Linaker, S. M. Sulaman, M. Höst, and R. M. de Mello, "Guidelines for conducting surveys in software engineering v. 1.1," *Lund University*, 2015.

[37] J. Bruin. (2011, Feb.) Ordinal logistic regression. [Online]. Available: https://stats.idre.ucla.edu/r/dae/ordinal-logistic-regression/

[38] J. Frost. Multicollinearity in regression analysis: Problems, detection, and solutions. [Online]. Available: https://statisticsbyjim.com/regression/multicollinearity-in-regression-analysis/

[39] D. Russo and K.-J. Stol, "Gender differences in personality traits of software engineers," *IEEE Transactions on Software Engineering*, 2020.

[40] P. E. Strandberg, T. J. Ostrand, E. J. Weyuker, W. Afzal, and D. Sundmark, "Intermittently failing tests in the embedded systems domain," *arXiv preprint arXiv:2005.06826*, 2020.

[41] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "Surveying the developer experience of flaky tests," *to appear*, 2022.

[42] "Sonarlint, code quality and code security starts in your ide," accessed: 2021–09–12. [Online]. Available: https://www.sonarlint.org/

[43] D. Hovemeyer and W. Pugh, "Finding bugs is easy," *Acm sigplan notices*, vol. 39, no. 12, pp. 92–106, 2004.

[44] A. Ahmad, O. Leifler, and K. Sandahl, "Empirical analysis of practitioners' perceptions of test flakiness factors," *Software Testing, Verification and Reliability*, vol. 31, no. 8, p. e1791, 2021.

Authorized licensed use limited to: ULAKBIM UASL - GAZI UNIV. Downloaded on November 17,2022 at 06:57:05 UTC from IEEE Xplore. Restrictions apply.